

Program:

- '''- Download a 2 new datasets related to greenskilling or sustainability*
- Import in python file*
- Note all the generalisations and analysis about the dataset, note relations of each feature/column to each other. (write it in text cells or comments)*
- if required do feature engineering*
- Handle null values*
- Report understanding of dataset*
- Used Seaborn library plots to visualise your dataset*
- Use scaling on numerical columns and pd.get_dummies / Label encoder on Categorical columns*
- Give your insights and answer why you did what you did.*
- Split data into train and test*
- Apply regression and classification algorithms*
- Evaluate your model respective to classification and regression problems*
- Save 2 files [1 for linear regression and 1 for Logistic regression] with name as today's date*
- Give insights and explain your models error values'''*

```
import matplotlib.pyplot as plt
import seaborn as s
import pandas as p
import joblib as j
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import StandardScaler, LabelEncoder, minmax_scale
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score,
accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
d1 = p.read_csv('processed_data.csv')
d2 = p.read_csv('green_skills_dataset.csv')
print('Dataset Overview:')
d2.info()
print('The student\'s CGPA greater than 7.5 are:')
print(d2[d2['CGPA'] > 7.5])
print('Number of females in the dataset:')
print(d2[d2['Gender'] == 'Female'])
print('The students having advanced python skills are:')
print(d2[d2['Python_Skill'] == 'Advanced'])
print('The number of null values in the dataset are:')
print(d2.isnull().sum().sum())
if d2.isnull().sum().sum() > 0:
    print('Handling null values by filling with mean for numerical columns and mode for
categorical columns.')
    for column in d2.columns:
        if d2[column].dtype == 'object':
            # Fill text columns with the most frequent value
            d2[column] = d2[column].fillna(d2[column].mode()[0])
```

```

else:
    # Fill numeric columns with the mean
    d2[column] = d2[column].fillna(d2[column].mean())
print('Visualizing the dataset using Seaborn library:')
print('Heatmap of the dataset:')
print('Scaling numerical columns and encoding categorical columns:')
print(minmax_scale(d2.select_dtypes(include=['float64', 'int64'])))
print('Standard Scaled Data:')
print(StandardScaler().fit_transform(
    d2.select_dtypes(include=['float64', 'int64'])))
corr = d2.corr(numeric_only=True)
s.heatmap(corr, annot=True)
plt.show()
print('Pairplot of the dataset:')
s.pairplot(d2)
plt.show()
print('Dummies for categorical columns:')
print('Dataset 2:')
d2 = p.get_dummies(d2, columns=['Stream', 'Gender', 'Python_Skill',
                                'AI_Knowledge', 'Cloud_Knowledge', 'Green_Certification'])

print(d2)
X1 = d1.drop("Electricity_Usage", axis=1)
y1 = d1["Electricity_Usage"]
X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y1,
    test_size=0.2,
    random_state=42
)
scaler = StandardScaler()
X1_train = scaler.fit_transform(X1_train)
X1_test = scaler.transform(X1_test)
le = LabelEncoder()
d2['Placement'] = le.fit_transform(d2['Placement'])
model = LinearRegression() # creating model object
model.fit(X1_train, y1_train) # Training the model
prediction = model.predict(X1_test) # Predicting the values
j.dump(model, '11-07-2026-linear.pkl')
print('Dataset1 linear regression evaluation metrics:')
print('Mean Absolute Error:')
print(mean_absolute_error(y1_test, prediction))
print('Mean Squared Error:')
print(mean_squared_error(y1_test, prediction))
print('R2 Score:')
print(r2_score(y1_test, prediction))
x2 = d2.drop('Placement', axis=1)
y2 = d2['Placement']

```

```

X2_train, X2_test, y2_train, y2_test = train_test_split(
    # training and testing data split: 80:20
    x2, y2, test_size=0.2, random_state=42)
X2_train = scaler.fit_transform(X2_train)
X2_test = scaler.transform(X2_test)
model = LogisticRegression()
model.fit(X2_train, y2_train)
prediction = model.predict(X2_test)
j.dump(model, '11-07-2026-logistic.pkl')
print('Dataset2 logistic regression evaluation metrics:')
print('Accuracy:')
print(accuracy_score(y2_test, prediction))
print('Precision:')
print(precision_score(y2_test, prediction, average='weighted'))
print('Recall:')
print(recall_score(y2_test, prediction, average='weighted'))
print('F1 Score:')
print(f1_score(y2_test, prediction, average='weighted'))
print('Confusion Matrix:')
print(confusion_matrix(y2_test, prediction))
print('Insights:')
print('1. This dataset 1 provides information about individual\'s daily lifestyle use.')
print('2. The dataset contains both numerical and categorical features.')
print('3. Dataset 2 contains sustainability-related information such as electricity usage, water usage, waste recycling, and eco-friendly practices.')
print('4. Missing values were handled using the mean for numerical columns and the mode for categorical columns to avoid losing valuable data.')
print('5. Categorical features were converted into numerical form using Label Encoding and One-Hot Encoding because machine learning models require numerical input.')
print('6. Numerical features were standardized using StandardScaler so that all features have a similar scale, improving model performance.')
print('7. Heatmaps helped identify correlations between numerical features, while pairplots showed relationships and data distributions.')
print('8. The dataset was split into 80% training data and 20% testing data to evaluate the model on unseen data.')
print('9. Linear Regression was used to predict the continuous value \'Electricity_Usage\', making it suitable for a regression problem.')
print('10. Logistic Regression was used to predict \'Placement\', which has two classes (Placed/Not Placed), making it a classification problem.')
print('11. The trained models were saved using Joblib so they can be reused without retraining.')
print('12. Regression performance was evaluated using MAE, MSE, and R2 Score, while classification performance was evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.')

```

Output:

Dataset Overview:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 14 entries, 0 to 13

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	Student_ID	10 non-null	float64
1	Stream	10 non-null	object
2	Gender	10 non-null	object
3	CGPA	10 non-null	float64
4	Python_Skill	10 non-null	object
5	AI_Knowledge	10 non-null	object
6	Cloud_Knowledge	10 non-null	object
7	Recycling_Knowledge	10 non-null	float64
8	Renewable_Energy	10 non-null	float64
9	Carbon_Footprint	10 non-null	float64
10	Green_Certification	10 non-null	object
11	Green_Skills_Score	10 non-null	float64
12	Placement	10 non-null	object

dtypes: float64(6), object(7)

memory usage: 1.6+ KB

The student's CGPA greater than 7.5 are:

	Student_ID	Stream	...	Green_Skills_Score	Placement
0	1001.0	CS	...	91.0	Yes
1	1002.0	IT	...	73.0	Yes
4	1005.0	Civil	...	87.0	Yes
5	1006.0	CS	...	96.0	Yes
6	1007.0	IT	...	69.0	Yes
8	1009.0	Mech	...	88.0	Yes

[6 rows x 13 columns]

Number of females in the dataset:

	Student_ID	Stream	...	Green_Skills_Score	Placement
0	1001.0	CS	...	91.0	Yes
2	1003.0	EC	...	48.0	No
4	1005.0	Civil	...	87.0	Yes
6	1007.0	IT	...	69.0	Yes
8	1009.0	Mech	...	88.0	Yes

[5 rows x 13 columns]

The students having advanced python skills are:

	Student_ID	Stream	...	Green_Skills_Score	Placement
0	1001.0	CS	...	91.0	Yes
5	1006.0	CS	...	96.0	Yes
8	1009.0	Mech	...	88.0	Yes

[3 rows x 13 columns]

The number of null values in the dataset are:

52

Handling null values by filling with mean for numerical columns and mode for categorical columns.

Scaling numerical columns and encoding categorical columns:

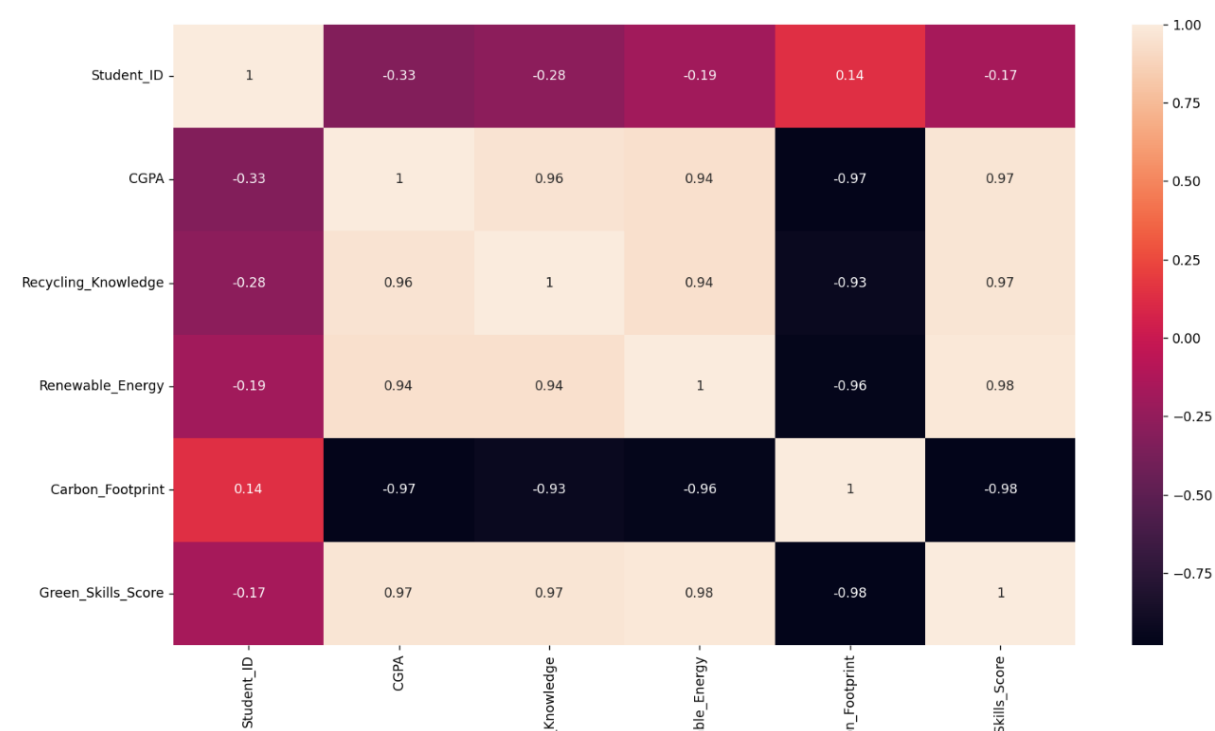
```
[[0.      0.89655172 0.83333333 0.83333333 0.16193656 0.90740741]
 [0.11111111 0.5862069 0.5      0.66666667 0.43238731 0.57407407]
 [0.22222222 0.27586207 0.16666667 0.16666667 0.81469115 0.11111111]
 [0.33333333 0.10344828 0.33333333 0.33333333 0.95492487 0.18518519]
 [0.44444444 0.75862069 0.66666667 1.      0.0884808 0.83333333]
 [0.55555556 1.      1.      1.      0.      1.      ]
 [0.66666667 0.44827586 0.5      0.5      0.39065109 0.5      ]
 [0.77777778 0.      0.      0.      1.      0.      ]
 [0.88888889 0.65517241 0.66666667 0.83333333 0.17863105 0.85185185]
 [1.      0.17241379 0.16666667 0.33333333 0.76961603 0.25925926]
 [0.5      0.48965517 0.48333333 0.56666667 0.47913189 0.52222222]
 [0.5      0.48965517 0.48333333 0.56666667 0.47913189 0.52222222]
 [0.5      0.48965517 0.48333333 0.56666667 0.47913189 0.52222222]
 [0.5      0.48965517 0.48333333 0.56666667 0.47913189 0.52222222]]
```

Standard Scaled Data:

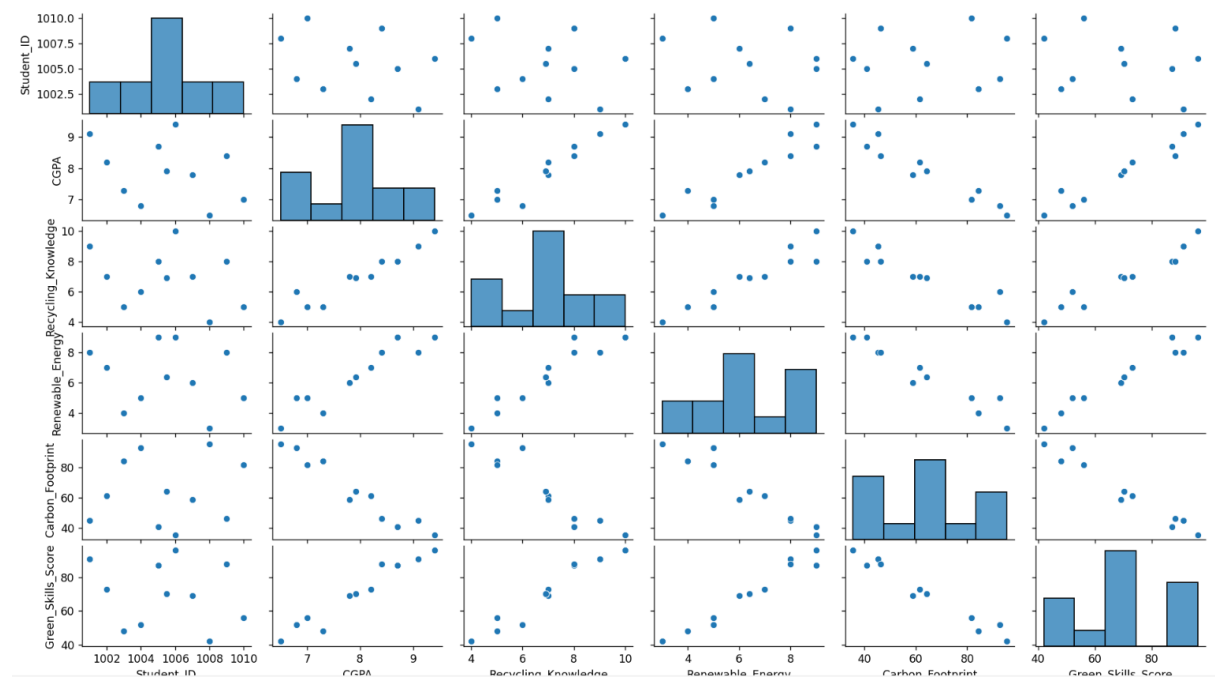
```
[[-1.85374314e+00 1.47041212e+00 1.36988895e+00 9.41875105e-01
 -1.04994125e+00 1.31035112e+00]
 [-1.44180022e+00 3.48911350e-01 6.52328073e-02 3.53203164e-01
 -1.54728184e-01 1.76393421e-01]
 [-1.02985730e+00 -7.72589419e-01 -1.23942334e+00 -1.41281266e+00
 1.11072732e+00 -1.39854784e+00]
 [-6.17914381e-01 -1.39564540e+00 -5.87095266e-01 -8.24140717e-01
 1.57491188e+00 -1.14655723e+00]
 [-2.05971460e-01 9.71967333e-01 7.17560880e-01 1.53054705e+00
 -1.29308554e+00 1.05836052e+00]
 [2.05971460e-01 1.84424571e+00 2.02221703e+00 1.53054705e+00
 -1.58596389e+00 1.62533938e+00]
 [6.17914381e-01 -1.49533436e-01 6.52328073e-02 -2.35468776e-01
 -2.92878349e-01 -7.55971803e-02]
 [1.02985730e+00 -1.76947899e+00 -1.89175141e+00 -2.00148460e+00
 1.72411406e+00 -1.77653374e+00]
 [1.44180022e+00 5.98133744e-01 7.17560880e-01 9.41875105e-01
 -9.94681186e-01 1.12135817e+00]
 [1.85374314e+00 -1.14642301e+00 -1.23942334e+00 -8.24140717e-01
 9.61525146e-01 -8.94566633e-01]
 [0.00000000e+00 -1.10676976e-15 -5.79383717e-16 -5.22845714e-16
 0.00000000e+00 -8.95250455e-16]
 [0.00000000e+00 -1.10676976e-15 -5.79383717e-16 -5.22845714e-16
 0.00000000e+00 -8.95250455e-16]]
```

```
[ 0.00000000e+00 -1.10676976e-15 -5.79383717e-16 -5.22845714e-16
 0.00000000e+00 -8.95250455e-16]
[ 0.00000000e+00 -1.10676976e-15 -5.79383717e-16 -5.22845714e-16
 0.00000000e+00 -8.95250455e-16]]
```

Visualizing the dataset using Seaborn library:
Heatmap of the dataset:



Pairplot of the dataset:



Dummies for categorical columns:

Dataset 2:

	Student_ID	CGPA	...	Green_Certification_No	Green_Certification_Yes
0	1001.0	9.10	...	False	True
1	1002.0	8.20	...	True	False
2	1003.0	7.30	...	True	False
3	1004.0	6.80	...	True	False
4	1005.0	8.70	...	False	True
5	1006.0	9.40	...	False	True
6	1007.0	7.80	...	True	False
7	1008.0	6.50	...	True	False
8	1009.0	8.40	...	False	True
9	1010.0	7.00	...	True	False
10	1005.5	7.92	...	True	False
11	1005.5	7.92	...	True	False
12	1005.5	7.92	...	True	False
13	1005.5	7.92	...	True	False

[14 rows x 23 columns]

Dataset1 linear regression evaluation metrics:

Mean Absolute Error:

53.93218419970553

Mean Squared Error:

3813.1180682742415

R2 Score:

0.47223279331844403

Dataset2 logistic regression evaluation metrics:

Accuracy:

1.0

Precision:

1.0

Recall:

1.0

F1 Score:

1.0

Confusion Matrix:

[[1 0]

[0 2]]

Insights:

1. This dataset 1 provides information about individual's daily lifestyle use.
2. The dataset contains both numerical and categorical features.
3. Dataset 2 contains sustainability-related information such as electricity usage, water usage, waste recycling, and eco-friendly practices.
4. Missing values were handled using the mean for numerical columns and the mode for categorical columns to avoid losing valuable data.

5. Categorical features were converted into numerical form using Label Encoding and One-Hot Encoding because machine learning models require numerical input.
6. Numerical features were standardized using StandardScaler so that all features have a similar scale, improving model performance.
7. Heatmaps helped identify correlations between numerical features, while pairplots showed relationships and data distributions.
8. The dataset was split into 80% training data and 20% testing data to evaluate the model on unseen data.
9. Linear Regression was used to predict the continuous value 'Electricity_Usage', making it suitable for a regression problem.
10. Logistic Regression was used to predict 'Placement', which has two classes (Placed/Not Placed), making it a classification problem.
11. The trained models were saved using Joblib so they can be reused without retraining.
12. Regression performance was evaluated using MAE, MSE, and R^2 Score, while classification performance was evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.